NUS National University of Singapore
Department of Statistics and Data Science
Faculty of Science

DSS5201 Data Visualisation

Week 9 Data Visualisation with `matplotlib`

David CHEW

Semester 2 AY25/26

Last Week: Principles of Visualisation

- Principles of Visualisation
- Graphical Excellence
- Best Practices & References

This Week: `matplotlib`

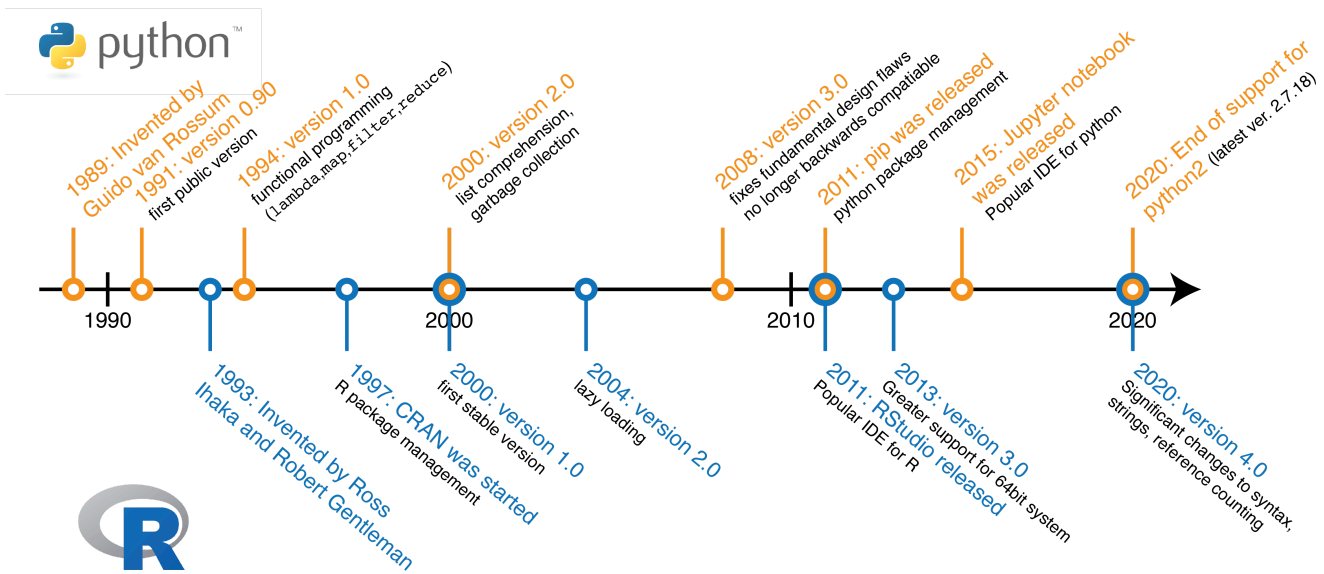
We will learn about the

- **object-oriented approach** for `matplotlib`
- histogram (`ax.hist`) — distribution of a continuous variable
- bar plot (`ax.barh`) — compares categories
- scatter plot (`ax.scatter`) — relationship between two continuous variables
- line plot (`ax.plot`) — visualises trends over time

Introduction

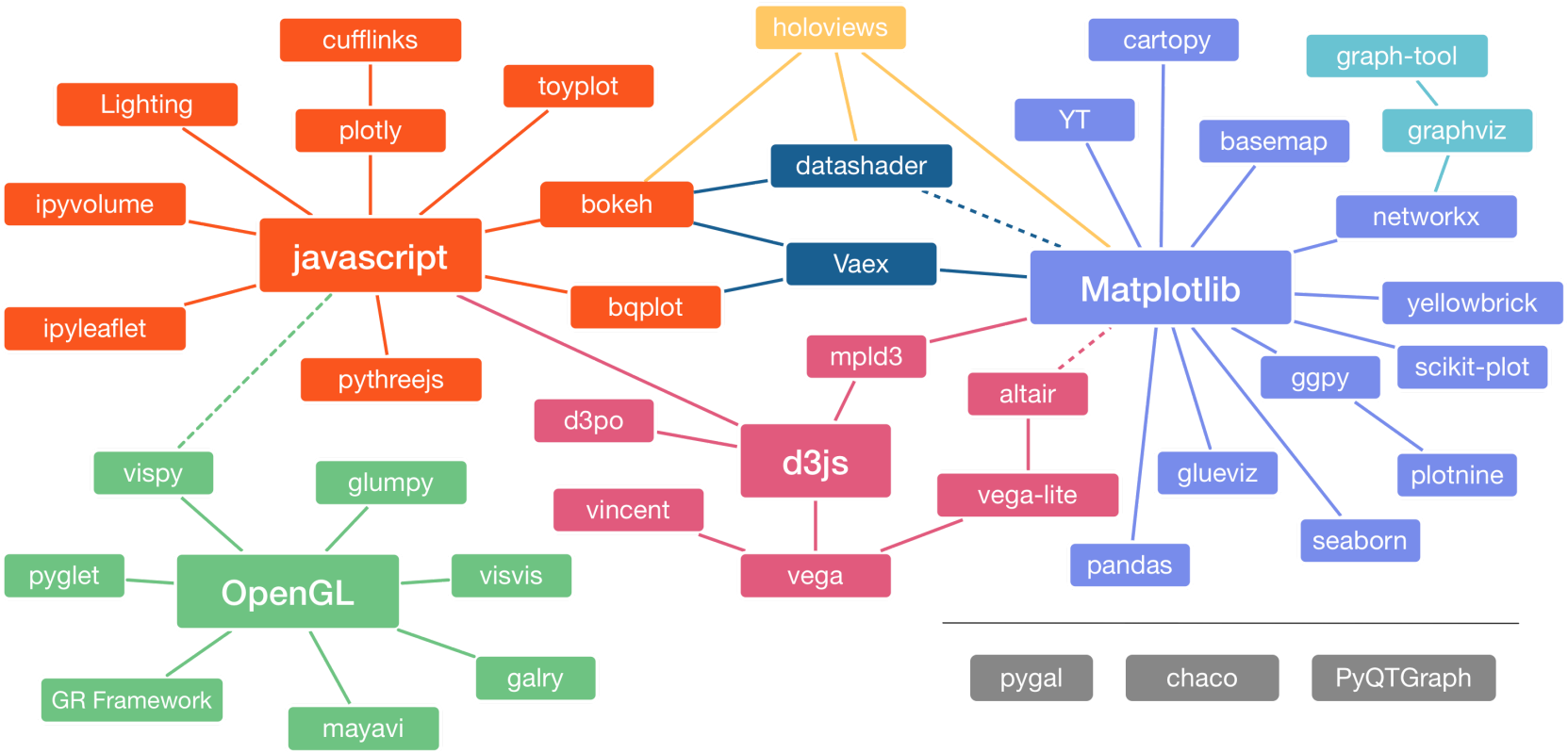
A Timeline of Python and R

Two popular programming languages in data science are Python and R.



Source: John F. Ouyang

Python Plotting Libraries



Source: Nicolas Rougier, adapted from Jake VanderPlas

Python Plotting Libraries

Basic

- `matplotlib`: Most fundamental plotting library.
  - Full control on a plot but can feel verbose.
  - Libraries built on it: `pandas`, `seaborn`, `plotnine`.

Intermediate

- `seaborn`: High-level interface for statistical graphics and tidy data.
- `plotnine`: Based on the Grammar of Graphics, similar to `R`'s `ggplot2`. Works well with tidy data.
- `plotly`: Enables interactive and web-based visualisations.

Financial Times Visual Vocabulary

Deviation

Emphasise variations (+/-) from a fixed reference point. Typically the reference point is zero but it can also be a target or a long-term average. Can also be used to show sentiment (positive/neutral/negative).

Example FT uses
Trade surplus/deficit; climate change

Diverging bar

A simple standard bar chart that can handle both negative and positive magnitude values.

Diverging stacked bar

Perfect for presenting survey results which involve sentiment (eg disagree/neutral/agree).

Spine

Splits a single value into two contrasting components (eg male/female).

Surplus/deficit filled line

The shaded area of these charts allows a balance to be shown – either against a baseline or between two series.

Correlation

Show the relationship between two or more variables. Be mindful that, unless you tell them otherwise, many readers will assume the relationships you show them to be causal (i.e. one causes the other).

Example FT uses
Inflation and unemployment, income and life expectancy

Scatterplot

The standard way to show the relationship between two continuous variables, each of which has its own axis.

Column + line timeline

A good way of showing the relationship between an amount (columns) and a rate (line).

Connected scatterplot

Usually used to show how the relationship between 2 variables has changed over time.

Bubble

Like a scatterplot, but adds additional detail by sizing the circles according to a third variable.

XY heatmap

A good way of showing the patterns between 2 categories of data, less effective at showing fine differences in amounts.

Ranking

Use where an item's position in an ordered list is more important than its absolute or relative value. Don't be afraid to highlight the points of interest.

Example FT uses
Wealth, deprivation, league tables, constituency election results

Ordered bar

Standard bar charts display the ranks of values much more easily when sorted into order.

Ordered column

See above.

Ordered proportional symbol

Use when there are big variations between values and/or seeing fine differences between data is not so important.

Dot strip plot

Dots placed in order on a strip are a space-efficient method of laying out ranks across multiple categories.

Slope

Perfect for showing how ranks have changed over time or vary between categories.

Lollipop

Lollipops draw more attention to the data value than standard bar/column and can also show rank and value effectively.

Bump

Effective for showing changing rankings across multiple dates. For large datasets, consider grouping lines using colour.

Distribution

Show values in a dataset and how often they occur. The shape (or 'skew') of a distribution can be a memorable way of highlighting the lack of uniformity or equality in the data.

Example FT uses
Income distribution, population (age/sex) distribution, revealing inequality

Histogram

The standard way to show a statistical distribution - keep the gaps between columns small to highlight the 'shape' of the data.

Dot plot

A simple way of showing the change or range (min/max) of data across multiple categories.

Dot strip plot

Good for showing individual values in a distribution, can be a problem when too many dots have the same value.

Barcode plot

Like dot strip plots, good for displaying all the data in a table, they work best when highlighting individual values.

Boxplot

Summarise multiple distributions by showing the median (centre) and range of the data

Violin plot

Similar to a box plot but more effective with complex distributions (data that cannot be summarised with simple average).

Population pyramid

A standard way for showing the age and sex breakdown of a population distribution; effectively, back to back histograms.

Cumulative curve

A good way of showing how unequal a distribution is: y axis is always cumulative frequency; x axis is always a measure.

Frequency polygons

For displaying multiple distributions of data. Like a regular line chart, best limited to a maximum of 3 or 4 datasets.

Beeswarm

Use to emphasise individual points in a distribution. Points can be sized to an additional variable. Best with medium-sized datasets

Change over Time

Give emphasis to changing trends. These can be short (intra-day) movements or extended series traversing decades or centuries. Choosing the correct time period is important to provide suitable context for the reader.

Example FT uses
Share price movements, economic time series, sectoral changes in a market

Line

The standard way to show a changing time series. If data are irregular, consider markers to represent data points.

Column

Columns work well for showing change over time - but usually best with only one series of data at a time.

Column + line timeline

A good way of showing the relationship over time between an amount (columns) and a rate (line).

Slope

Good for showing changing data as long as the data can be simplified into 2 or 3 points without missing a key part of story.

Area chart

Use with care – these are good at showing changes to total, but seeing change in components can be very difficult.

Candlestick

Usually focused on day-to-day activity, these charts show opening/closing and high/low points of each day.

Fan chart (projections)

Use to show the uncertainty in future projections – usually this grows the further forward to projection.

Connected scatterplot

A good way of showing changing data for two variables whenever there is a relatively clear pattern of progression.

Calendar heatmap

A great way of showing temporal patterns (daily, weekly, monthly) – at the expense of showing precision in quantity.

Priestley timeline

Great when date and duration are key elements of the story in the data.

Circle timeline

Good for showing discrete values of varying size across multiple categories (eg earthquakes by continent).

Vertical timeline

Presents time on the Y axis. Good for displaying detailed time series that work especially well when scrolling on mobile.

Seismogram

Another alternative to the circle timeline for showing series where there are big variations in the data.

Streamgraph

A type of area chart; use when seeing changes in proportions over time is more important than individual values

Magnitude

Show size comparisons. These can be relative (just being able to see larger/bigger) or absolute (need to see fine differences). Usually these show a 'counted' number (for example, barrels, dollars or people) rather than a calculated rate or per cent.

Example FT uses
Commodity production; market capitalisation, volumes in general

Column

The standard way to compare the size of things. Must always start at 0 on the axis.

Bar

See above. Good when the data are not time series and labels have long category names.

Paired column

As per standard column but allows for multiple series. Can become tricky to read with more than 2 series.

Paired bar

See above.

Marimekko

A good way of showing the size and proportion of data at the same time – as long as the data are not too complicated.

Proportional symbol

Use when there are big variations between values and/or seeing fine differences between data is not so important.

Isotype (pictogram)

Excellent solution in some instances – use only with whole numbers (do not slice off an arm to represent a decimal).

Lollipop

Lollipop charts draw more attention to the data value than standard bar/column – does not have to start at zero (but preferable).

Radar

A space-efficient way of showing value of multiple variables– but make sure they are organised in a way that makes sense to reader.

Parallel coordinates

An alternative to radar charts – again, the arrangement of the variables is important. Usually benefits from highlighting values.

Bullet

Good for showing a measurement against the context of a target or performance range.

Grouped symbol

An alternative to bar/column charts when being able to count data or highlight individual elements is useful.

Part-to-whole

Show how a single entity can be broken down into its component elements. If the reader's interest is solely in the size of the components, consider a magnitude-type chart instead.

Example FT uses
Fiscal budgets, company structures, national election results

Stacked column/bar

A simple way of showing part-to-whole relationships but can be difficult to read with more than a few components.

Marimekko

A good way of showing the size and proportion of data at the same time – as long as the data are not too complicated.

Pie

A common way of showing part-to-whole data – but be aware that it's difficult to accurately compare the size of the segments.

Donut

Similar to a pie chart – but the centre can be a good way of making space to include more information about the data (eg total).

Treemap

Use for hierarchical part-to-whole relationships; can be difficult to read when there are many small segments.

Voronoi

A way of turning points into areas – any point within each area is closer to the central point than any other centroid.

Arc

A hemicycle, often used for visualising parliamentary composition by number of seats.

Gridplot

Good for showing % information, they work best when used on whole numbers and work well in small multiple layout form.

Venn

Generally only used for schematic representation.

Waterfall

Can be useful for showing part-to-whole relationships where some of the components are negative.

Spatial

Aside from locator maps only used when precise locations or geographical patterns in data are more important to the reader than anything else.

Example FT uses
Population density, natural resource locations, natural disaster risk/impact, catchment areas, variation in election results

Basic choropleth (rate/ratio)

The standard approach for putting data on a map – should always be rates rather than totals and use a sensible base geography.

Proportional symbol (count/magnitude)

Use for totals rather than rates – be wary that small differences in data will be hard to see.

Flow map

For showing unambiguous movement across a map.

Contour map

For showing areas of equal value on a map. Can use deviation colour schemes for showing +/- values

Equalised cartogram

Converting each unit on a map to a regular and equally-sized shape – good for representing voting regions with equal value.

Scaled cartogram (value)

Stretching and shrinking a map so that each area is sized according to a particular value.

Dot density

Used to show the location of individual events/locations – make sure to annotate any patterns the reader should see.

Heat map

Grid-based data values mapped with an intensity colour scale. As choropleth map – but not snapped to an admin/political unit.

Flow

Show the reader volumes or intensity of movement between two or more states or conditions. These might be logical sequences or geographical locations.

Example FT uses
Movement of funds, trade, migrants, lawsuits, information; relationship graphs.

Sankey

Shows changes in flows from one condition to at least one other; good for tracing the eventual outcome of a complex process.

Waterfall

Designed to show the sequencing of data through a flow process, typically budgets. Can include +/- components.

Chord

A complex but powerful diagram which can illustrate 2-way flows (and net winner) in a matrix.

Network

Used for showing the strength and inter-connectedness of relationships of varying types.

Visual vocabulary

Designing with data

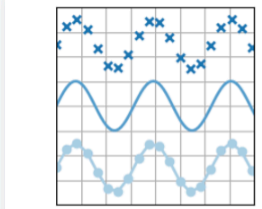
There are so many ways to visualise data - how do we know which one to pick? Use the categories across the top to decide which data relationship is most important in your story, then look at the different types of chart within the category to form some initial ideas about what might work best. This list is not meant to be exhaustive, nor a wizard, but is a useful starting point for making informative and meaningful data visualisations.

FT graphic: Alan Smith, Chris Campbell, Ian Bott, Liz Faunce, Graham Parrish, Billy Ehrenberg-Shannon, Paul McCallum, Martin Stabe
Inspired by the Graphic Continuum by Jon Schwabish and Severino Ribecca

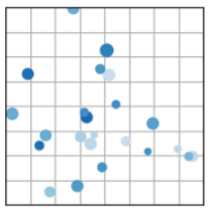
ft.com/vocabulary

Object-Oriented Approach to `matplotlib`

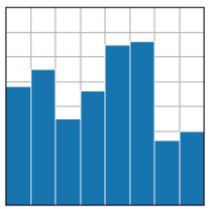
Visualisation with `matplotlib`



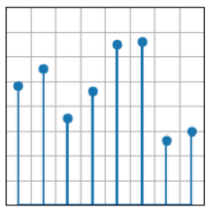
plot(x, y)



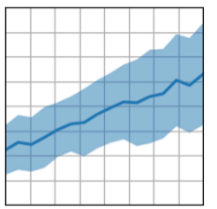
scatter(x, y)



bar(x, height)



stem(x, y)



fill_between(x, y1, y2)

`matplotlib` is a fundamental Python library for 2D graphics.

- Initially developed to replicate MATLAB's plotting interface.
- Gradually evolved into a flexible, publication-quality visualization library.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
```

Two Approaches to `matplotlib`

1. **The `MATLAB` (`pyplot`) approach:**

- Uses simple functions like `plt.plot()`.
- Inspired by `MATLAB`'s style.
- Convenient, but may not scale well for complex or multi-panel figures.

Example:

```
plt.plot(x, y)
plt.title("Sample plot")
plt.show()
```

2. **The modern object-oriented approach:**

- Uses `fig, ax = plt.subplots()` and methods like `ax.plot()`.
- Provides full control over figure elements.
- Preferred for customization, multi-panel layouts.

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("Sample plot")
plt.show()
```

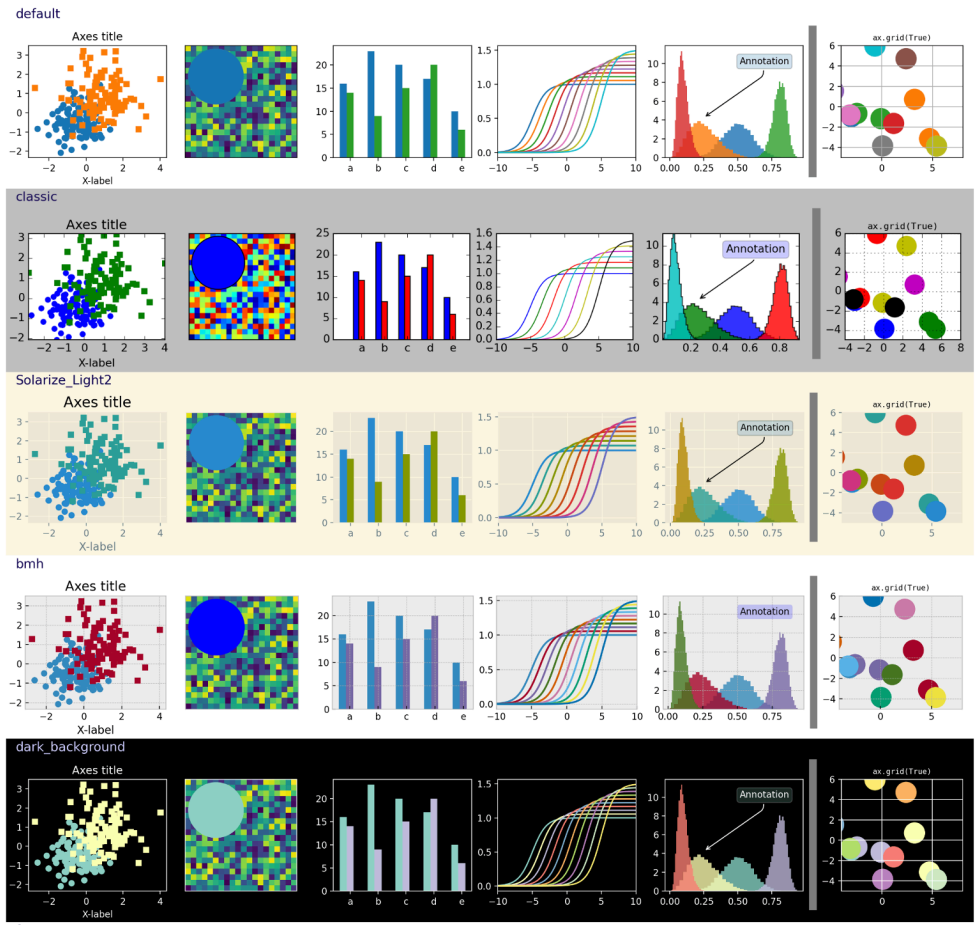
`matplotlib` Style Sheets

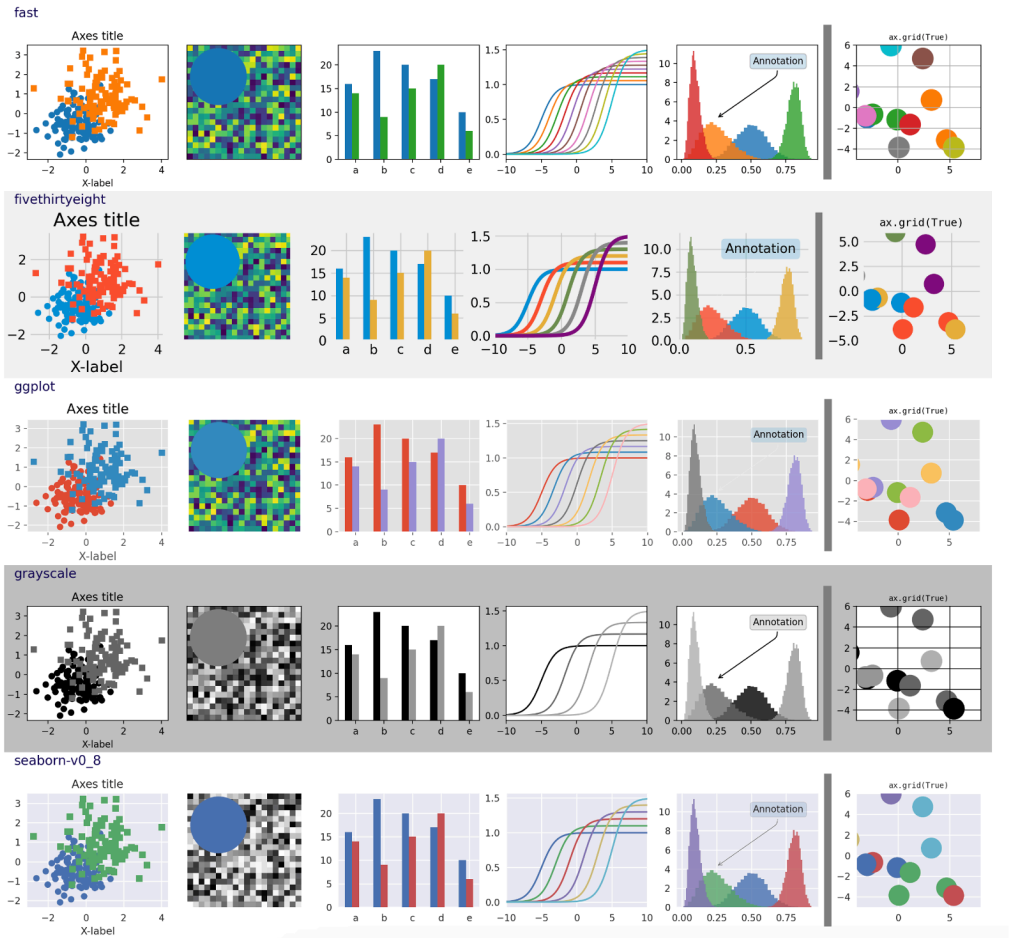
- `matplotlib` comes with built-in style sheets that control the overall look of the plots.
- A consistent theme makes our visuals look cohesive and professional across our presentation or report.

```
1 print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic',
'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'petroff10', 'seaborn-v0_8', 'seaborn-
v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-
darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper',
'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-
white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

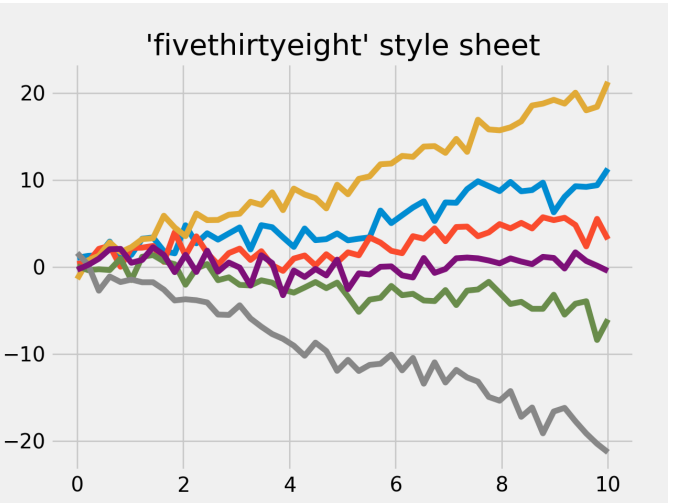
matplotlib Style Sheets





Setting a Style: `fivethirtyeight`

Lets us quickly change the visual theme and use the `fivethirtyeight` style sheet.

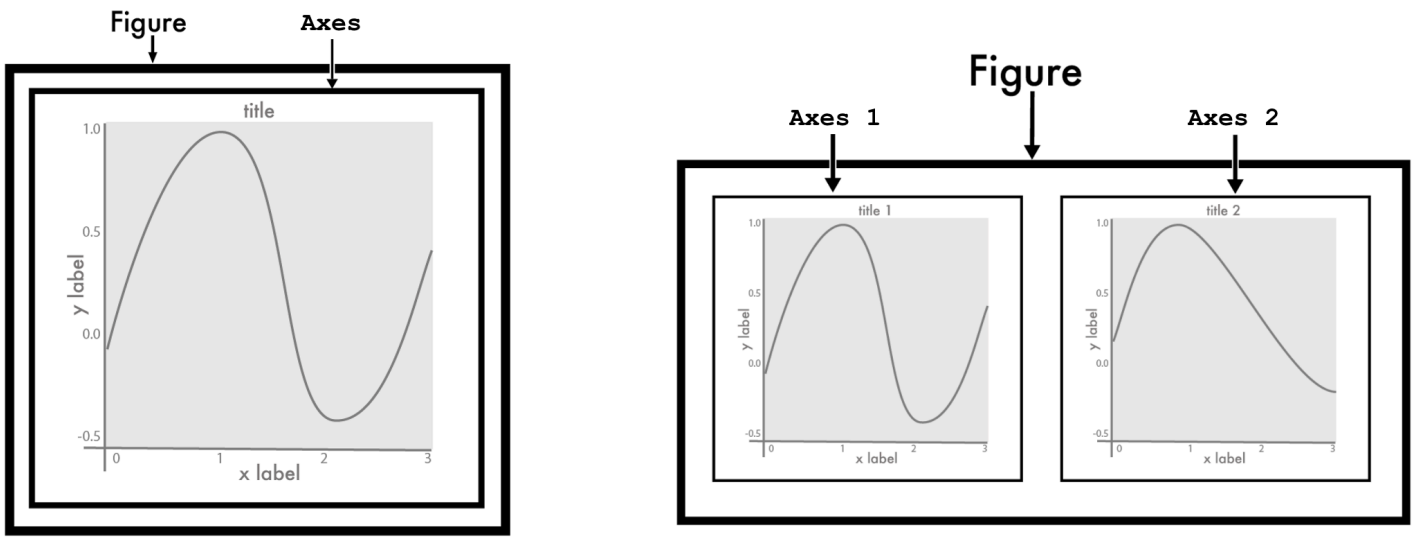


- `fivethirtyeight` is a journalistic theme that is great for storytelling and reports.
  - Light gray background.
  - Subtle white grid lines.
  - Thicker lines and larger fonts.
- Set the theme with `plt.style.use()`. It will be applied to all subsequent plots.

```
1 plt.style.use('fivethirtyeight')
```

Object-Oriented Approach

Key Terminologies

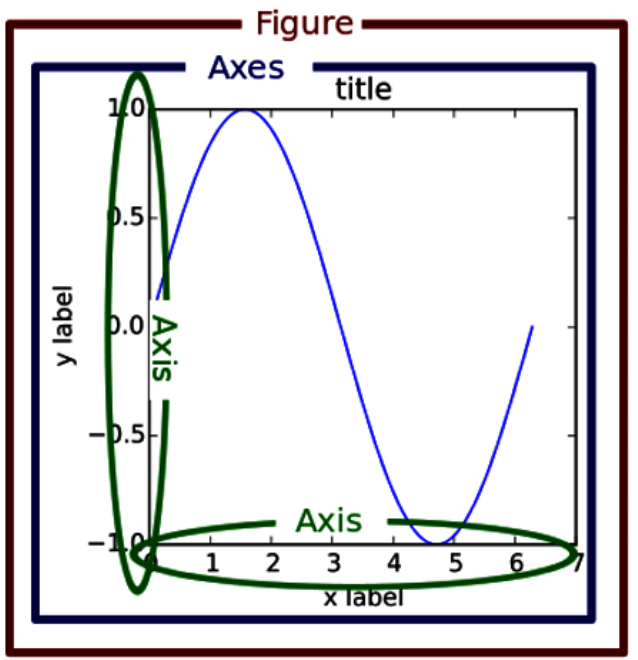


The `matplotlib` object-oriented approach is based on two key objects:

- The **figure** is the entire visualization and can contain one or multiple subplots.
- Each **axes** represents one (sub)plot inside the **figure** object.

Object-Oriented Approach

Key Terminologies



In addition to the **figure** and **axes**, there is one more important term:

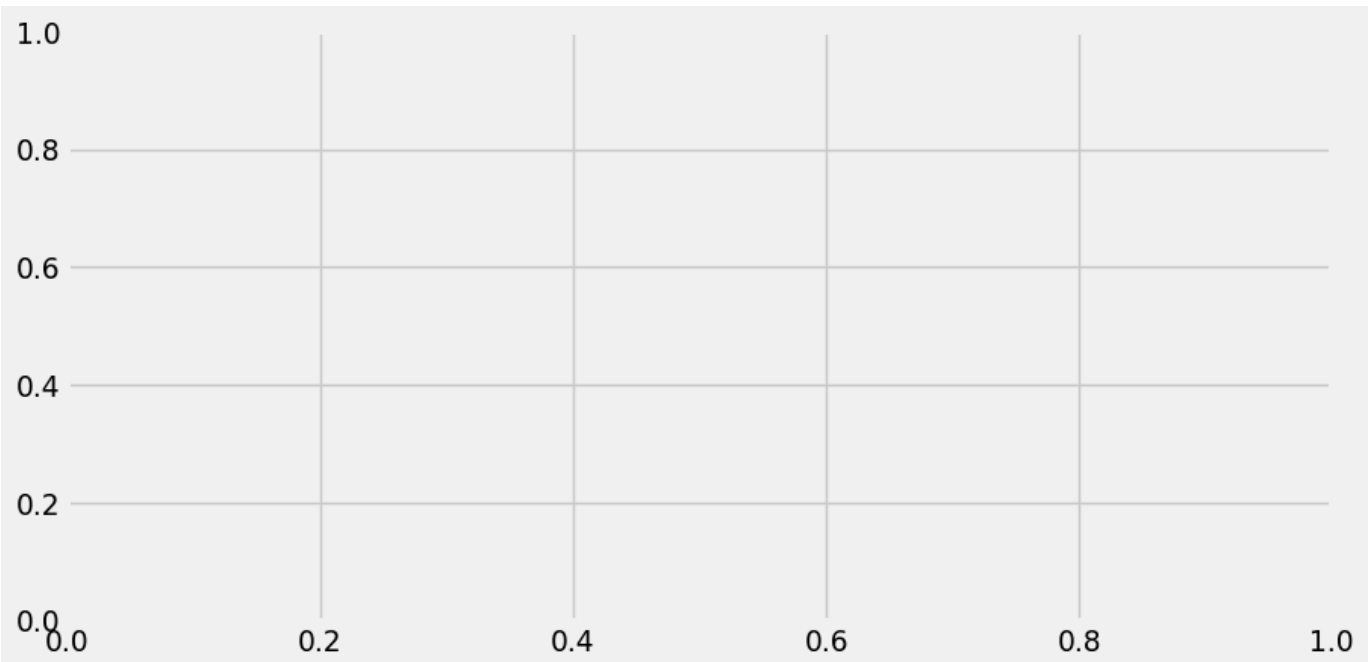
- The **axis** refers to an individual axis (x-axis or y-axis) within a specific **axes** object.

Object-Oriented Approach

Getting Started

- Use `plt.subplots()` to create the **figure** (`fig`) and at least one **axes** (`ax`).
- The command below creates a blank figure with one empty plotting area.

```
fig, ax = plt.subplots()
plt.show()
```

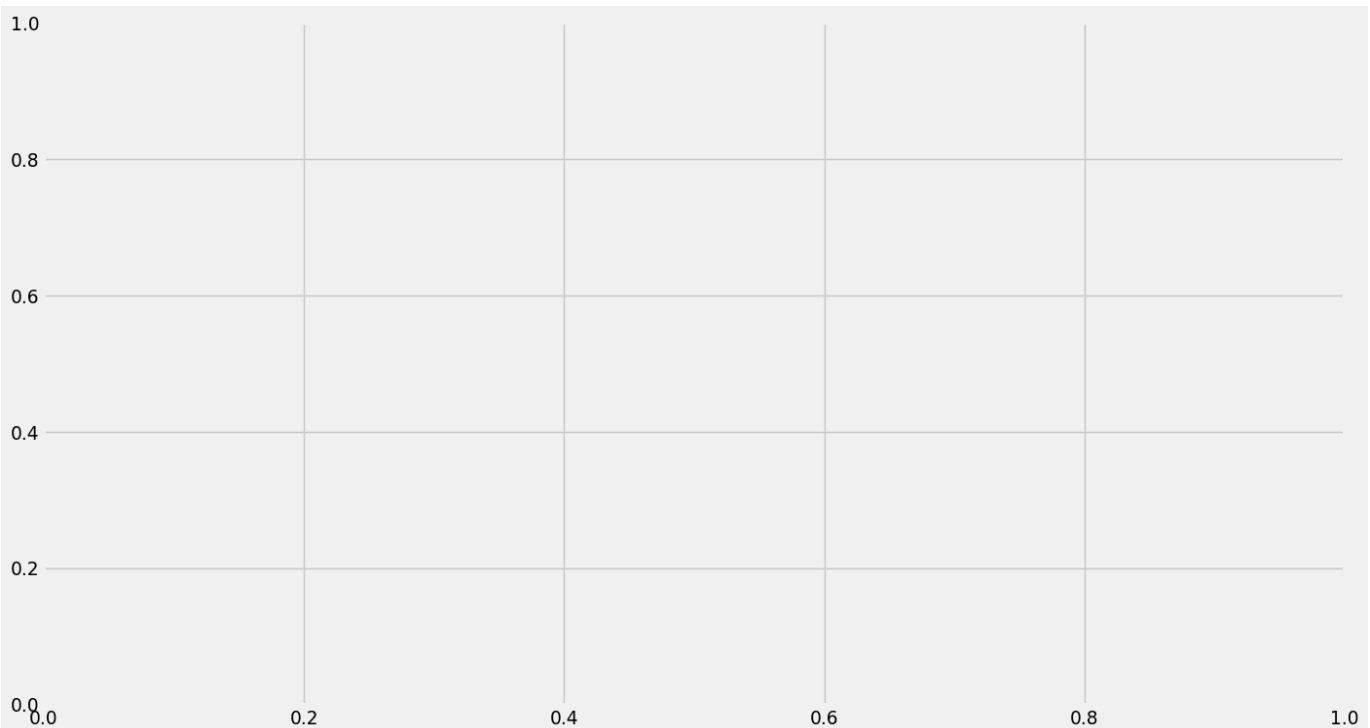


Object-Oriented Approach

Getting Started

- Both `fig` and `ax` are created at the same time.
- We can define the figure size with `figsize = (width, height)`.

```
1 fig, ax = plt.subplots(figsize = (16, 9))
2 plt.show()
```



Histograms

Earnings Data

Let's revisit data from `wk4_earnings.csv`.

```
from itables import show
df = pd.read_csv('../data/wk4_earnings.csv', header = 0)
df['sex'] = df['sex'].str.lower()
df['earnings'] = df['earnings']/1000
show(df, pageLength = 5, layout={"topStart": None}, searching=False)
```

height	sex	edu	age	earnings
189	male	16	45	50.000
166	female	16	58	60.000
162	female	16	29	30.000
160	female	16	91	50.000
161	female	17	39	51.000

Showing 1 to 5 of 1,192 entries

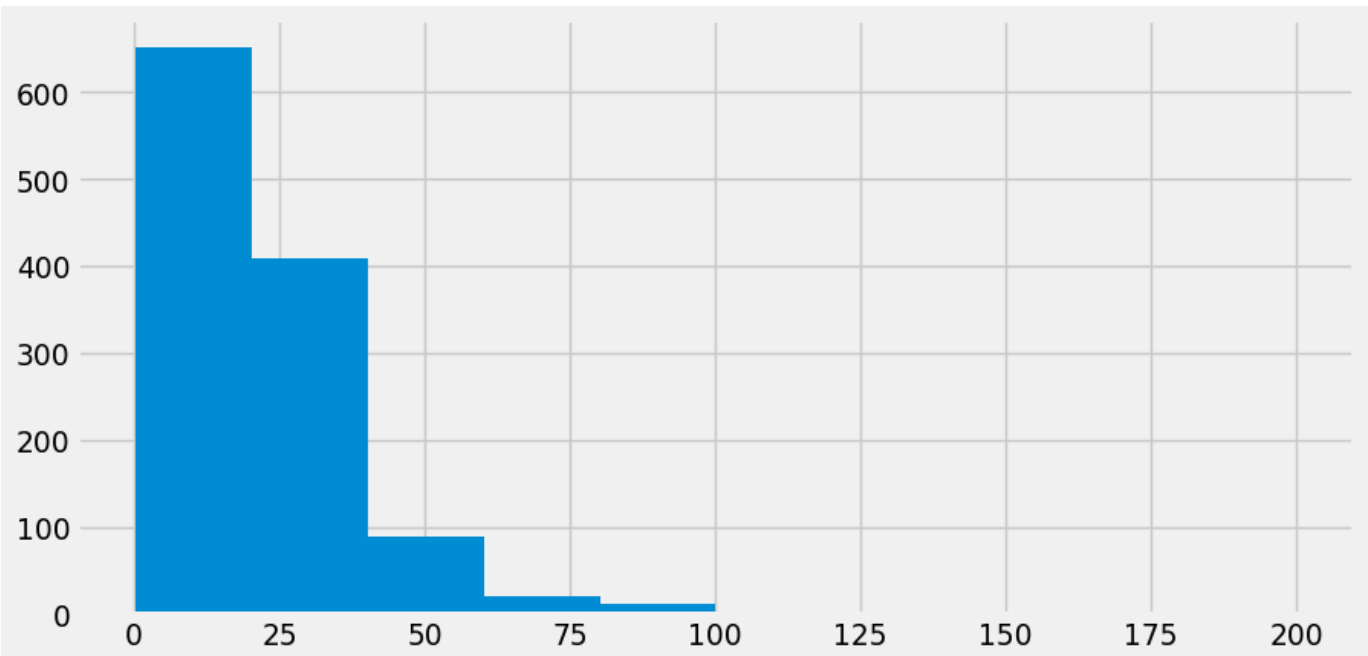
« ‹ 1 2 3 4 5 … 239 › »

Earnings Data

Histogram

We can add data to the plot using `ax` and create a **histogram** for `earnings`:

```
fig, ax = plt.subplots(figsize = (10, 5))
ax.hist(df['earnings'])
plt.show()
```

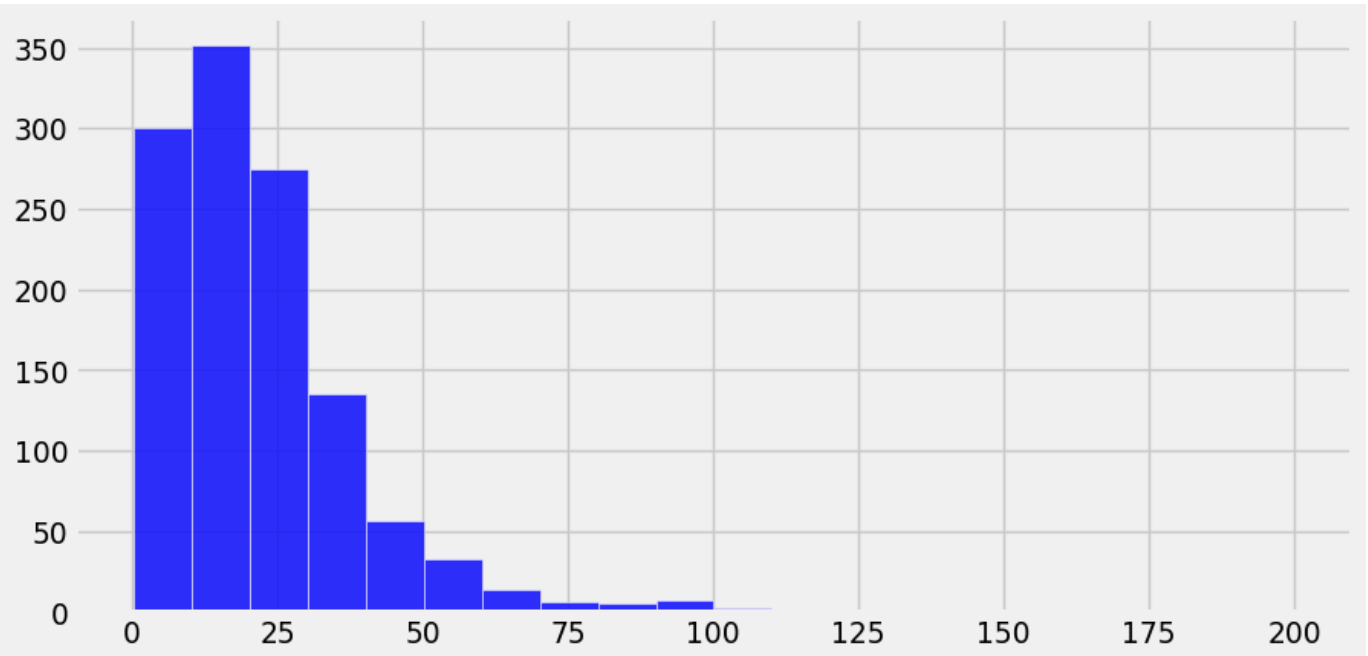


Earnings Data

Histogram

Now adjust the number of bins and appearance of the histogram:

```
fig, ax = plt.subplots(figsize = (10, 5))
ax.hist(df['earnings'], bins = 20, alpha = .8, color = 'b', edgecolor = 'w')
plt.show()
```

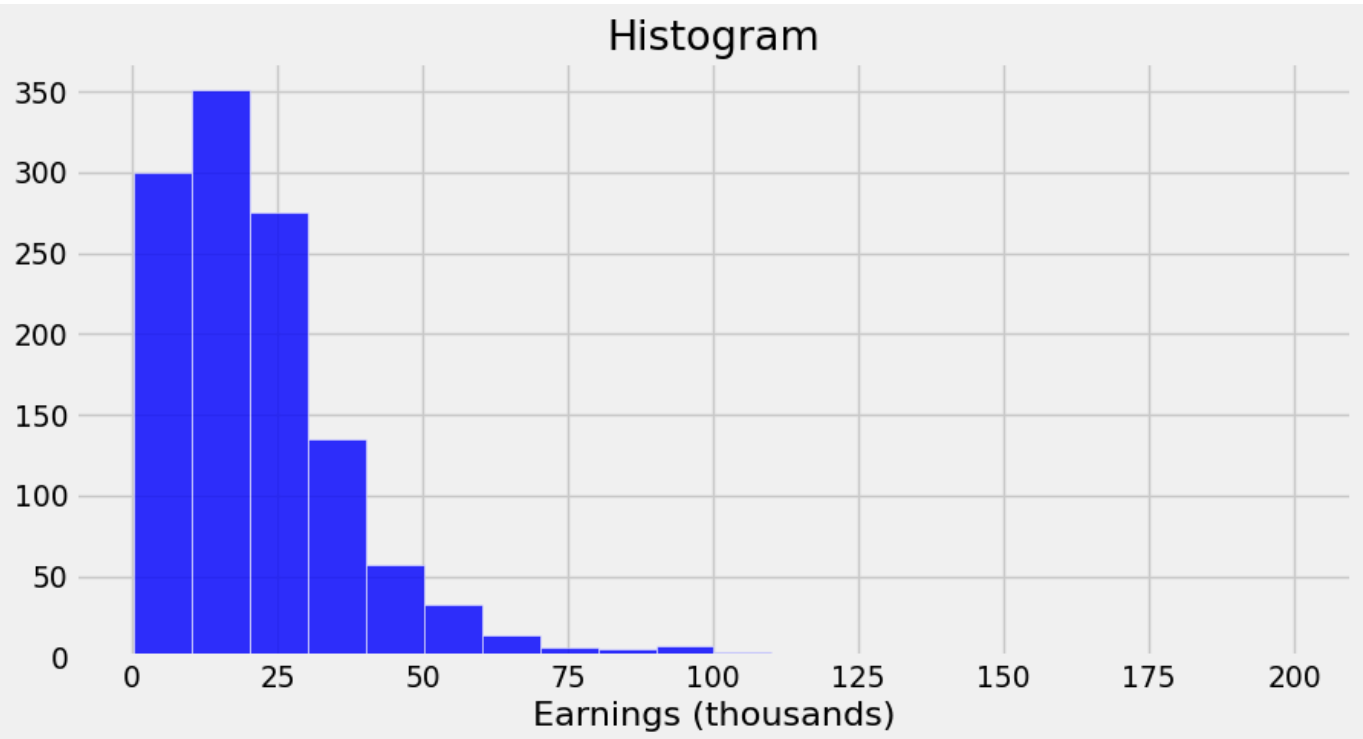


Earnings Data

Histogram

Add title and axis labels with `ax.set()`:

```
fig, ax = plt.subplots(figsize = (10, 5))
ax.hist(df['earnings'], bins = 20, alpha = .8, color = 'b', edgecolor = 'w')
ax.set(title = 'Histogram', xlabel = 'Earnings (thousands)')
plt.show()
```

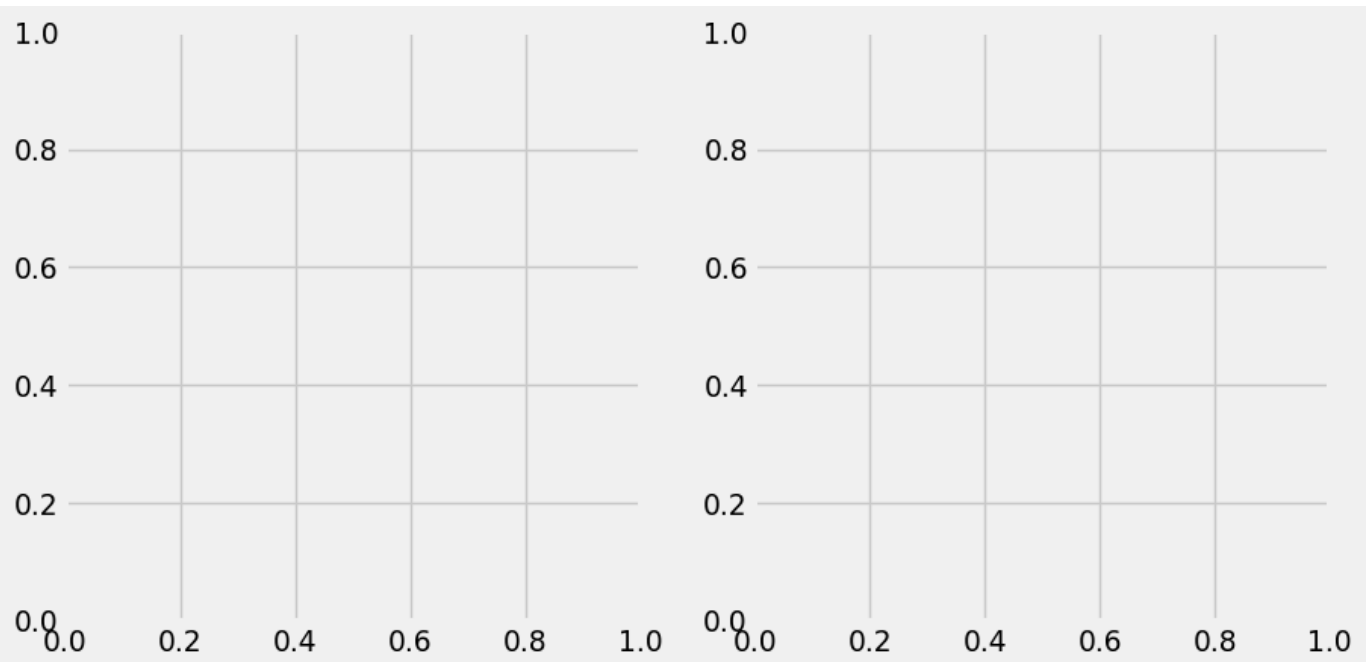


Multi-Panel Figure

`matplotlib`'s object-oriented approach allows multiple subplots in a figure.

- Each subplot is represented by an independent **axes** object.
- The following creates a 1 by 2 grid of subplots:

```
1 fig, (ax1, ax2) = plt.subplots(1, 2)
```

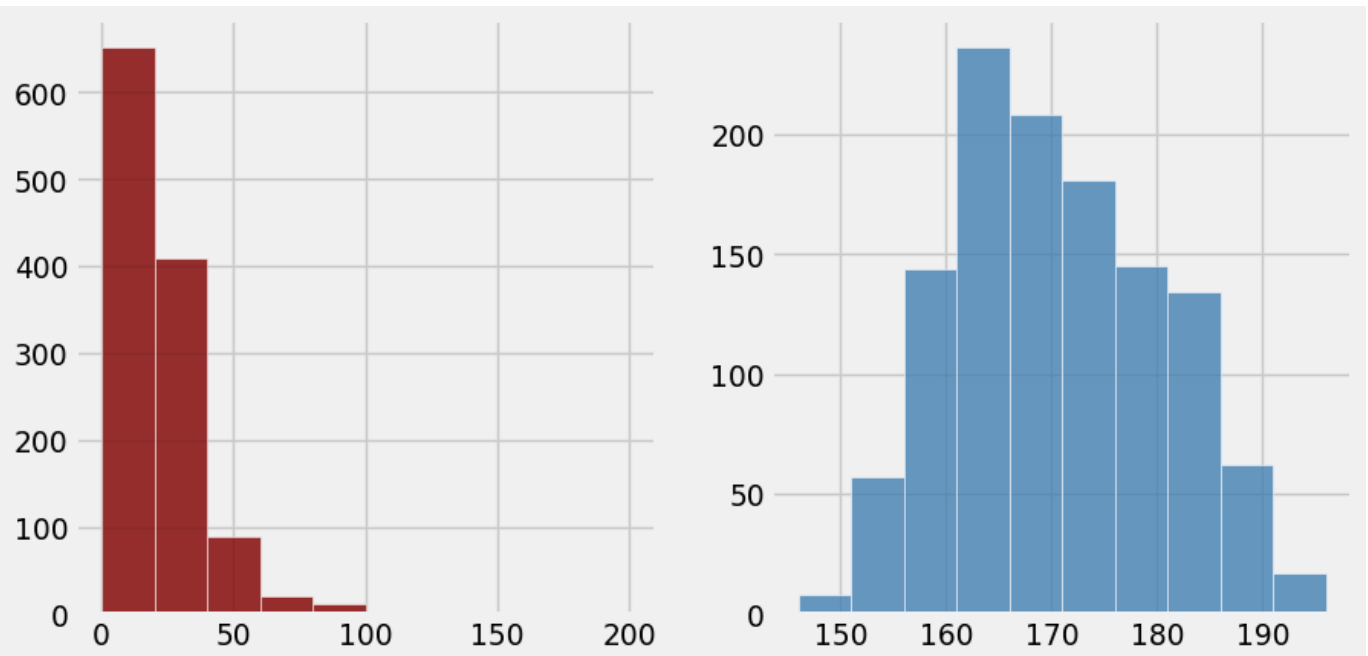


Earnings Data

Histograms for `earnings` and `height`

We create two histograms, one for `earnings` (`ax1`) and another for `height` (`ax2`).

```
fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.hist(df['earnings'], alpha = .8, color = 'maroon',    edgecolor = 'w')
ax2.hist(df['height'],   alpha = .8, color = 'steelblue', edgecolor = 'w')
plt.show()
```



Earnings Data

Histograms for `earnings` and `height`

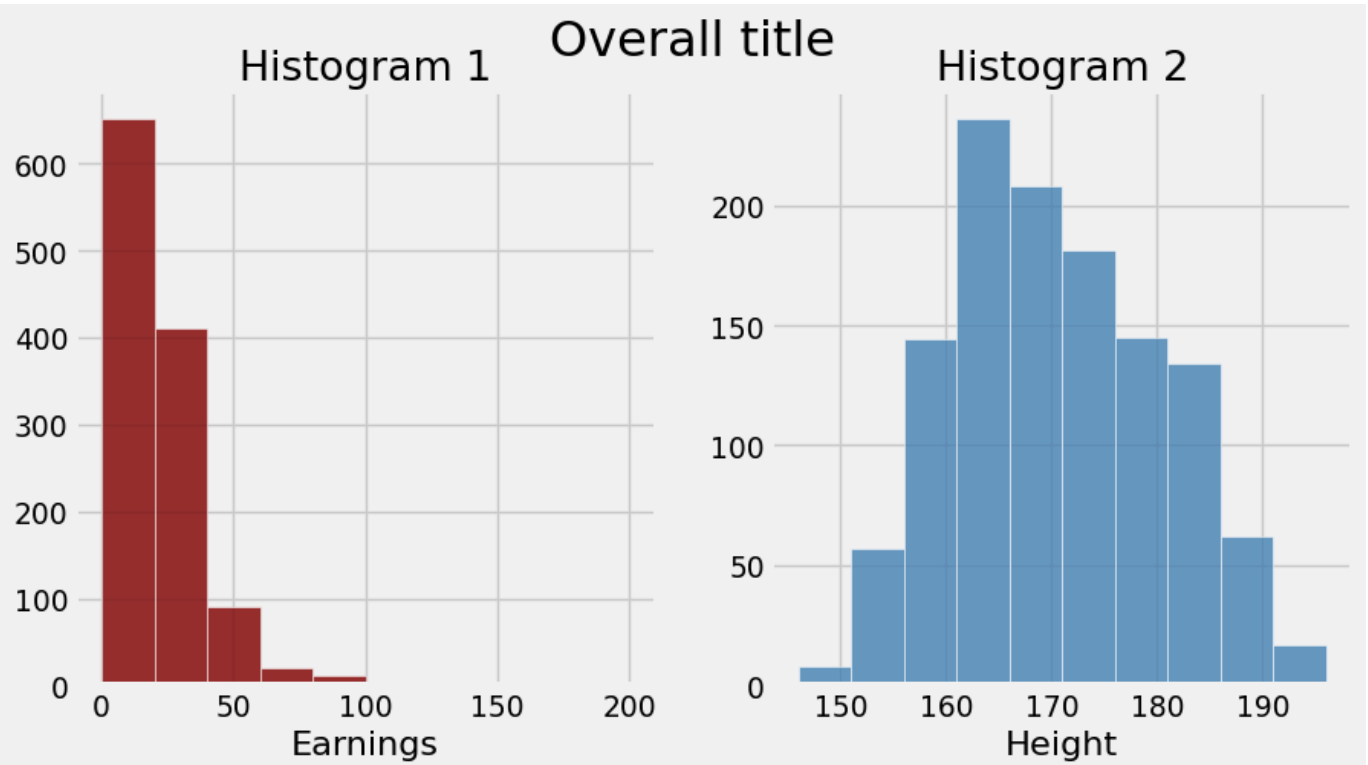
Each subplot (`ax1` and `ax2`) has its own **axes** object.

- `ax1.set()` and `ax2.set()` define the title and labels of the subplots.
- `fig.suptitle()` adds the overall title of the entire `figure`.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize = (10, 5))
fig.suptitle('Overall title', fontsize = 24)
ax1.hist(df['earnings'], alpha = .8, color = 'maroon',    edgecolor = 'w')
ax2.hist(df['height'],   alpha = .8, color = 'steelblue', edgecolor = 'w')
ax1.set(title = 'Histogram 1', xlabel = 'Earnings')
ax2.set(title = 'Histogram 2', xlabel = 'Height')
plt.show()
```

Earnings Data

Histograms for `earnings` and `height`



Earnings Data

Saving the Plot

We can export the plot as an image file.

- `dpi = 80` controls the image resolution.
- `bbox_inches = 'tight'` trims extra white space around the figure.

The following saves the image in the `src` folder:

```
1 fig.savefig('myplot.png', dpi = 80, bbox_inches = 'tight')
```

Bar Plots

Nobel Prize Data

Every October, the Nobel Prize Committee awards prizes in science, literature, economics, and peace.

- We will use the data from 1901 to 2016 and explore historical patterns and trends.
- Data set: `wk10_nobel_prizes16.csv`

Nobel Prize Data

```
1 nobel = pd.read_csv('../data/wk10_nobel_prizes16.csv')
2 show(nobel, layout={"topStart": None}, searching=False)
```

laureateID	category	fullName	year	age	gender	born
160	Chemistry	Jacobus H. van 't Hoff	1901	49.0	Male	the N
569	Literature	Sully Prudhomme	1901	62.0	Male	Fran
293	Medicine	Emil von Behring	1901	47.0	Male	Pola
463	Peace	Frederic Passy	1901	73.0	Male	Fran
462	Peace	Henry Dunant	1901	73.0	Male	Swit:
1	Physics	Wilhelm Conrad Rontgen	1901	56.0	Male	Gerr
161	Chemistry	Emil Fischer	1902	50.0	Male	Gerr
571	Literature	Theodor Mommsen	1902	85.0	Male	Gerr
294	Medicine	Ronald Ross	1902	45.0	Male	Indi:
464	Peace	Elie Ducommun	1902	69.0	Male	Swit:

Showing 1 to 10 of 911 entries

« ‹ 1 2 3 4 5 … 92 › »

Nobel Prize Data

We look at a few more ways to visualise this data with `matplotlib`:

- **Distribution and deviation:** Histogram and bar plot
- **Changes over time:** Line plot (with shaded regions)
- **Correlation:** Scatter plot

Let us also switch to the `ggplot` stylesheet:

```
plt.style.use('ggplot')
```

Nobel Prize Data

Overall Winners

Question: Which countries have produced the most Nobel Prize winners?

- We begin by summarizing the data to understand the patterns.
- Here `.value_counts()` summarizes the number of Nobel laureates' by country of birth.

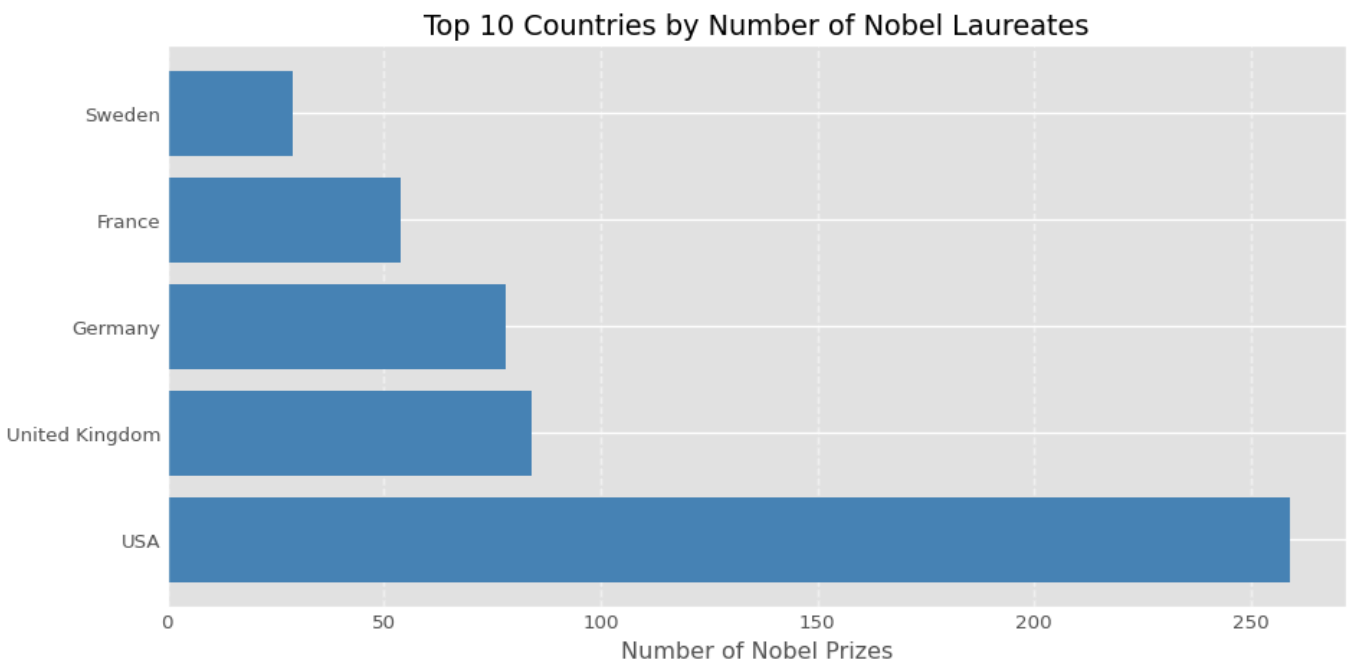
```
df1 = nobel['bornCountry'].value_counts().reset_index().head(5)
df1
```

	bornCountry	count
0	USA	259
1	United Kingdom	84
2	Germany	78
3	France	54
4	Sweden	29

Nobel Prize Data

Overall Winners: Bar Plot

```
fig, ax = plt.subplots(figsize = (10, 5))
ax.barh(df1['bornCountry'], df1['count'], color = 'steelblue')
ax.set(xlabel = 'Number of Nobel Prizes', ylabel = '',
       title = 'Top 10 Countries by Number of Nobel Laureates')
ax.grid(axis = 'x', linestyle = '--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Nobel Prize Data

Repeat Winners

Question: Who won the Nobel Prize more than once?

```
df_repeat = nobel['laureateID'].value_counts().reset_index()
df_repeat = df_repeat[df_repeat['count'] > 1]
repeat_winners = df_repeat.merge(
    nobel[['laureateID', 'fullName']],
    on = 'laureateID',
    how = 'left').drop_duplicates('laureateID')
repeat_winners
```

	laureateID	count	fullName
0	482	3	International Committee of the Red Cross
3	515	2	Office of the United Nations High Commissioner...
5	66	2	John Bardeen
7	217	2	Linus Pauling
9	222	2	Frederick Sanger
11	6	2	Marie Curie

Nobel Prize Data

Repeat Winners: Table

- Our goal is to communicate the insights efficiently.
- When the data is simple and specific, it is easier to present it as a table, instead of a plot.

```
df2 = repeat_winners[['fullName', 'count']]
df2 = df2.rename(str.title, axis = 'columns')
df2.style.hide(axis = 'index')
```

Fullname	Count
International Committee of the Red Cross	3
Office of the United Nations High Commissioner for Refugees	2
John Bardeen	2
Linus Pauling	2
Frederick Sanger	2
Marie Curie	2

Line Plots

Nobel Prize Data

Number of Winners Across Years

Question: How has the number of Nobel Prize winners evolved over time?

```
df3 = nobel.groupby('year').size().reset_index(name = 'count')
df3.head()
```

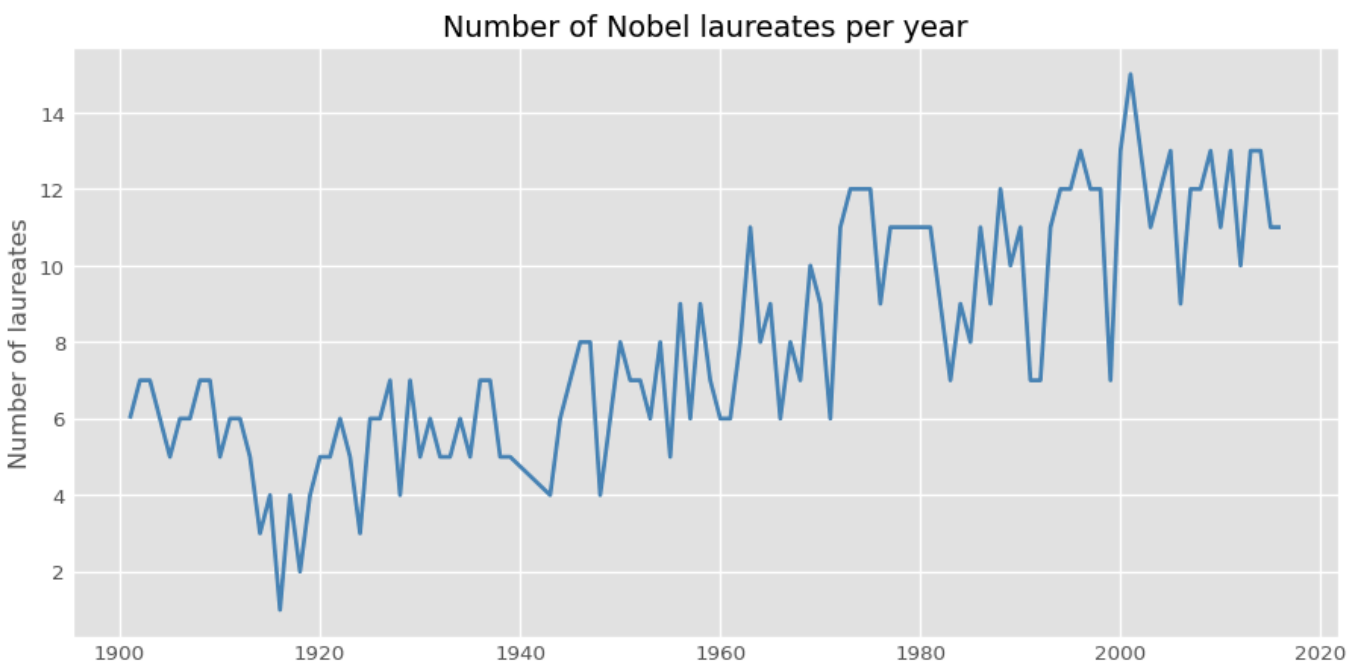
	year	count
0	1901	6
1	1902	7
2	1903	7
3	1904	6
4	1905	5

Nobel Prize Data

Number of Winners Across Years: Line Plot

A line plot works well to help us see the long-term patterns.

```
fig, ax = plt.subplots(figsize = (10, 5))
ax.plot(df3['year'], df3['count'], color = 'steelblue', linewidth = 2)
ax.set(ylabel = 'Number of laureates',
       title = 'Number of Nobel laureates per year')
plt.show()
```



Nobel Prize Data

Number of Winners Across Years: Smooth Line with Rolling Average

- To focus on the broader trend, we **smooth** the line using a rolling average.
- `.rolling()` creates a moving window across the data. The following calculates the mean for the current year ± 2 years on each side.

```
df3['rolling'] = df3['count'].rolling(5, center = True).mean()
df3.head()
```

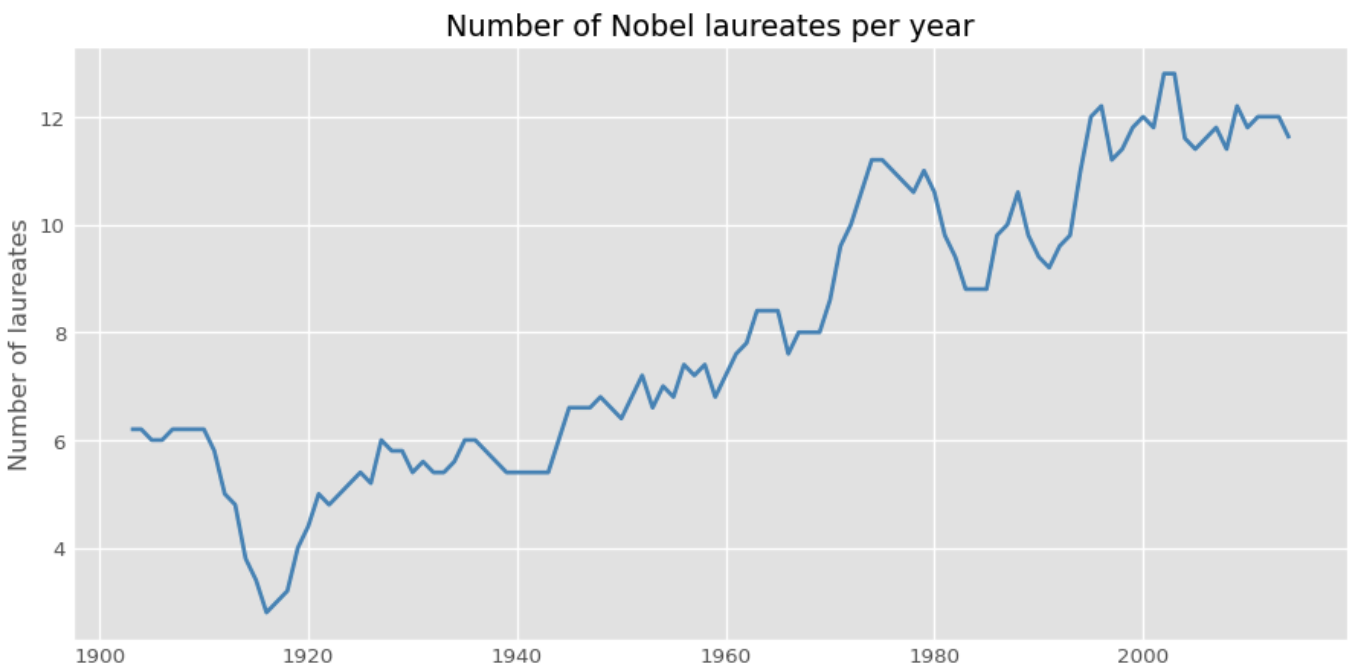
	year	count	rolling
0	1901	6	NaN
1	1902	7	NaN
2	1903	7	6.2
3	1904	6	6.2
4	1905	5	6.0

Nobel Prize Data

Number of Winners Across Years: Smooth Line with Rolling Average

Here's the smoothed line plot:

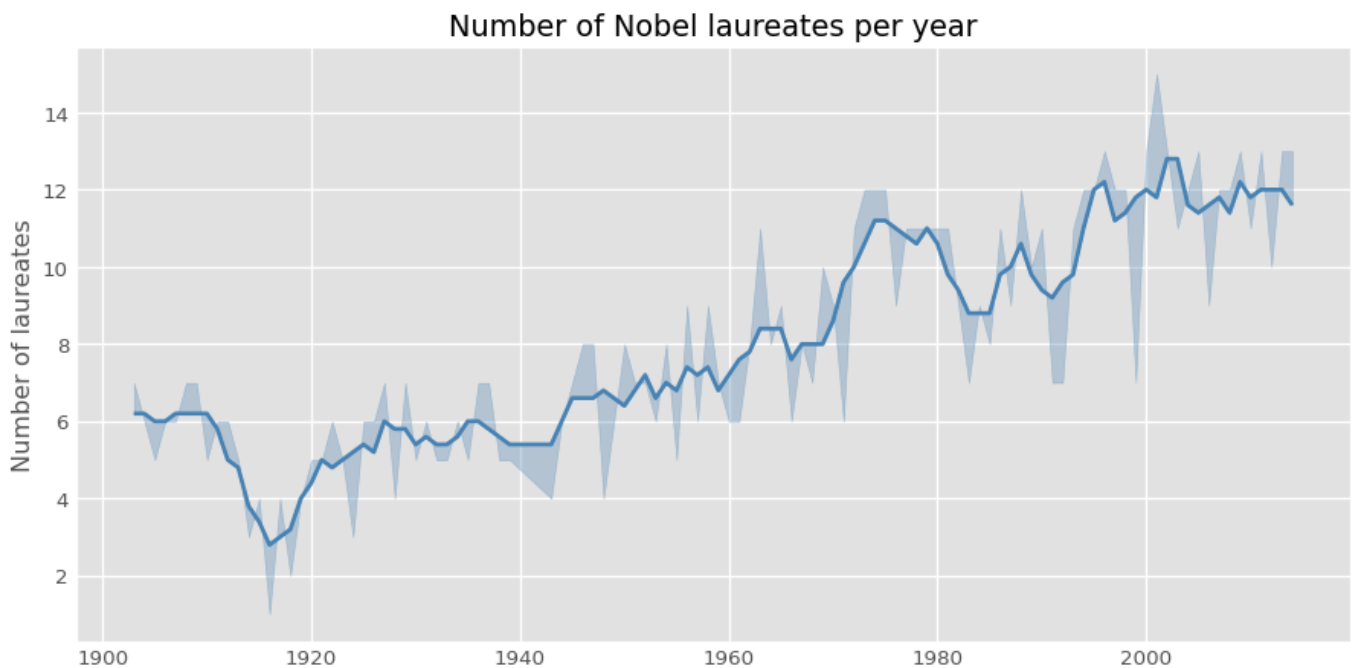
```
fig, ax = plt.subplots(figsize = (10, 5))
ax.plot(df3['year'], df3['rolling'], color = 'steelblue', linewidth = 2)
ax.set(ylabel = 'Number of laureates',
       title = 'Number of Nobel laureates per year')
plt.show()
```



Nobel Prize Data

Number of Winners Across Years: `ax.fill_between`

- So far, we've used a line to show the overall smooth trend.
- To highlight the variations over time, we can shade the area between the actual counts and the 5-year rolling averages.



Nobel Prize Data

Number of Winners Across Years: `ax.fill_between`

- `ax.fill_between()` takes three inputs: `x`, `y1`, and `y2`.
- They define the area between the actual counts and the rolling average.
- The shaded region can help readers quickly spot where the data fluctuates above or below the long-term trend.

```
fig, ax = plt.subplots(figsize = (10, 5))
ax.plot(df3['year'], df3['rolling'],
        color = 'steelblue', linewidth = 2)
ax.fill_between(df3['year'], df3['count'], df3['rolling'],
                color = 'steelblue', alpha = 0.3)
ax.set(ylabel = 'Number of laureates',
       title = 'Number of Nobel laureates per year')
plt.show()
```

Scatter Plots

Nobel Prize Data

Age of Winners Across Fields

Question: Do Nobel winners receive awards at similar ages across fields?

- Before plotting, we subset the data to include individual laureates only.
- We also extract the unique prize categories so we can visualize each one in a separate subplot.

```
df4 = nobel[nobel['gender'].isin(['Female', 'Male'])]
categories = df4['category'].unique()
categories
```

```
array(['Chemistry', 'Literature', 'Medicine', 'Peace', 'Physics',
       'Economics'], dtype=object)
```

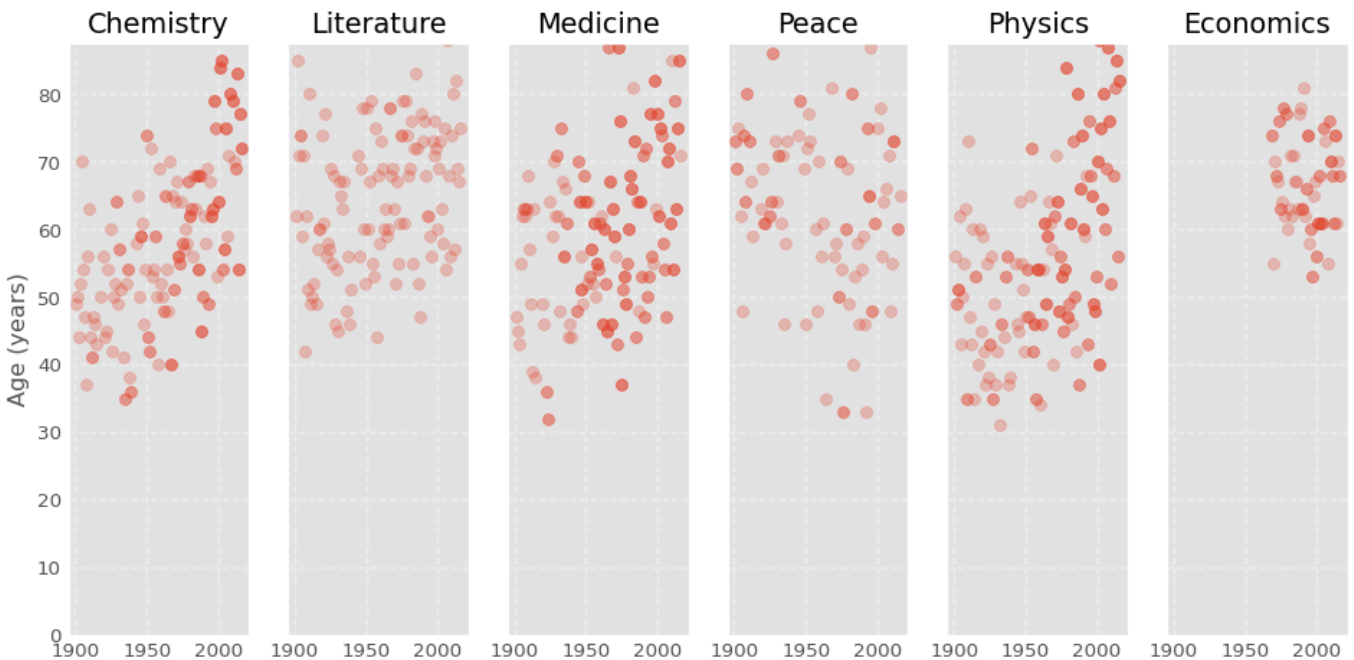
Nobel Prize Data

Age of Winners Across Fields: Scatter Plot Panels

```
fig, axes = plt.subplots(1, 6, figsize = (10, 5),
                         sharex = True, sharey = True)
for ax, cat in zip(axes, categories):
    sub = df4[df4['category'] == cat]
    ax.scatter(sub['year'], sub['age'], alpha = 0.3)
    ax.set(title = cat)
    ax.grid(True, linestyle = '--', alpha = 0.3)
    ax.set_ylim(bottom = 0)
axes[0].set_ylabel('Age (years)')
plt.tight_layout()
plt.show()
```

Nobel Prize Data

Age of Winners Across Fields: Scatter Plot Panels



Nobel Prize Data

Age of Winners Across Fields: Scatter Plot Panels

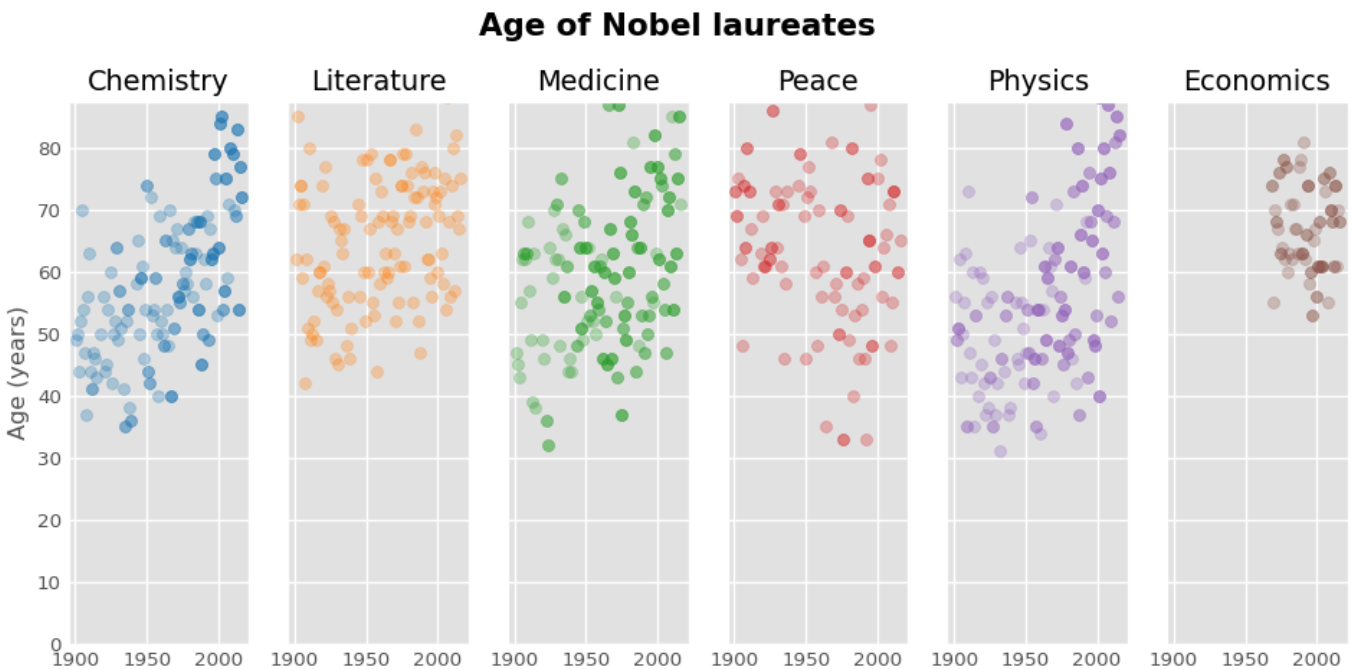
We add colors for distinct categories:

```
cmap = plt.get_cmap('tab10').colors
fig, axes = plt.subplots(1, 6, figsize = (10, 5),
                         sharex = True, sharey = True)
fig.suptitle('Age of Nobel laureates',
             fontsize = 16, fontweight = 'bold')
for ax, cat, colors in zip(axes, categories, cmap):
    sub = df4[df4['category'] == cat]
    ax.scatter(sub['year'], sub['age'], alpha = 0.3, color = colors)
    ax.set_title(cat)
    ax.set_ylim(bottom = 0)
axes[0].set_ylabel('Age (years)')
plt.tight_layout()
plt.show()
```

Note: Read more about `get_cmap` here.

Nobel Prize Data

Age of Winners Across Fields: Scatter Plot Panels



Summary & References

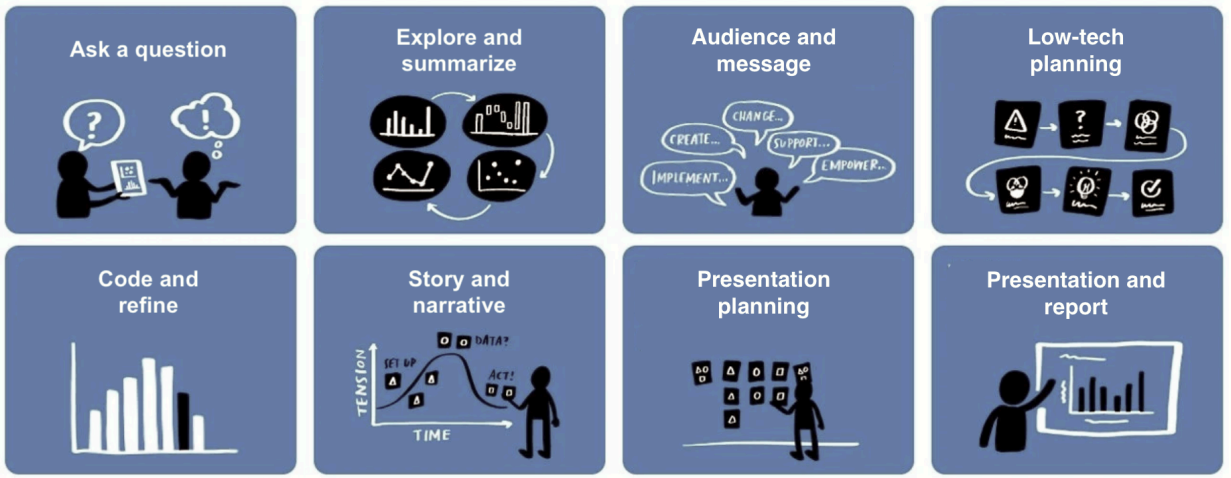
Summary

We learned about the

- **object-oriented approach** for `matplotlib`
- histogram (`ax.hist`) — distribution of a continuous variable
- bar plot (`ax.barh`) — compares categories
- scatter plot (`ax.scatter`) — relationship between two continuous variables
- line plot (`ax.plot`) — visualises trends over time

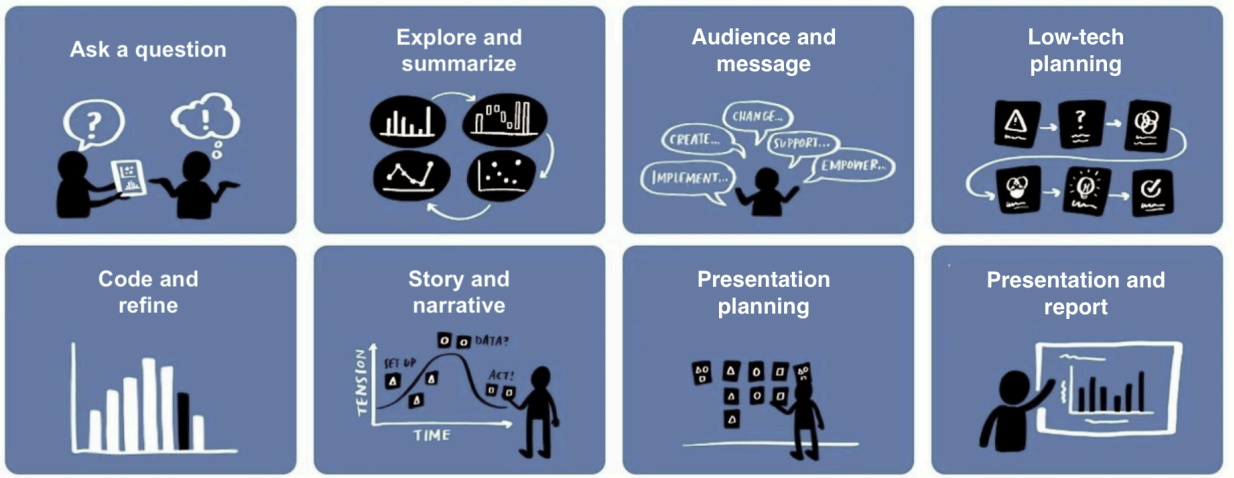
Additional resource: `matplotlib` color maps.

Reference: Exploration and Insights



1. Ask a question.
    - We can begin with a broad question (e.g., who won the Nobel Prizes). Then refine it as we explore the data.
2. Prepare and summarize data.
    - Clean, subset, compute summaries as preparation for the data exploration.

Reference: Exploration and Insights



3. Visualize patterns.
    - Choose plots that are suitable for your question, e.g., distribution, trend, and comparison.
4. Refine and communicate the insights.
    - Provide relevant context (do some background research if needed), highlight key findings, and interpret the overall patterns.
    - Connect your results back to the question.

Adapted from Storytelling with Data